

Shared Augmented Reality Experience Between a Microsoft Flight Simulator User and a User in the Real World

Christoph Leuze^{1*}, Matthias Leuze²

¹Nakamir Inc, Palo Alto, CA USA ²Alpinschule Innsbruck, Innsbruck, AT

ABSTRACT

Our demo consists of an application that allows a user with an augmented reality (AR) display to watch another user, flying an airplane in the Microsoft Flight Simulator 2020 (MSFS), at their respective location in the real world. To do that, we take the location data of a virtual 3D airplane model in a virtual representation of the world of a user playing MSFS, and stream it via a server to a mobile AR device. The mobile device user can then see the same 3D airplane model at exactly that real world location, that corresponds to the location of the virtual 3D airplane model in the virtual representation of the world. The mobile device user can also see the plane movement updated according to the 3D airplane movement in the virtual world. We implemented the application on both a cellphone and a see-through headset.

Keywords: outdoor, parallel reality, game, shared experience

Index Terms: Human-centered computing - Collaborative and social computing - Collaborative and social computing theory, concepts and paradigms - Social content sharing

1 INTRODUCTION

In many applications the real world position of a user is streamed to software applications to provide the real world user with additional information about their location. E.g. the real world position of a car is visualized as a little symbol on a software map application to aid with pathfinding. More recently the real world location of users can be used to allow users to interact with a virtual representation of the world, e.g., in Pokemon Go (Niantic, CA, USA), where the real world location decides, which virtual places the user can currently interact with or what types of creatures the user can see in the virtual world.

The other way around, however, has not yet been done or shown in software. A user that only interacts in the virtual world, e.g., by flying an airplane in a flight simulator, driving a car on a virtual 3D rendering of a real world race track or walking around in a 3D rendering of a real world building, can currently not be seen in the real world and no application exists that show a person in the real world which users in the virtual world currently preside at, or close to, their real world location.

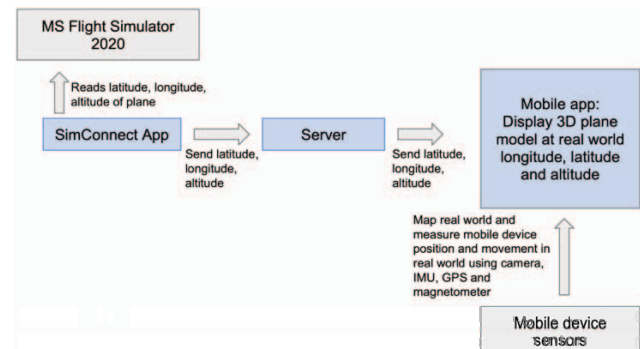
In this application, we are showing for the first time how it is possible to connect a virtual representation of the world with the real world and stream data from a user, who moves and interacts only in the virtual world, to the real world so that that an avatar of the user can be seen by people in the real world with the help of a mobile device.

Specifically, we are showing an example of a user flying an airplane in the Microsoft Flight Simulator 2020 (MSFS) and making this airplane visible in the real world via an AR application on a cell phone and a see-through headset.

2 METHOD

Our software application consists of three parts (Figure 1):

1. an application (SimConnect App) that reads the position information of a virtual airplane in the MSFS and sends it to our server,
2. a server that receives the position information and sends it to a mobile device and
3. an AR application running on a mobile device that connects to the server and receives the position information of the virtual airplane, maps out the real world space to understand its own position in the real world and visualizes a 3D model of the virtual airplane in the real world via the mobile device's screen.



of a virtual airplane in the flight simulator at a defined frequency, the Server, which receives this position information and sends it out to a mobile device, and the mobile application that calculates, based on the mobile device sensors (camera, IMU, GPS, magnetometer) and the received position, the 3D airplane model location and movement in the real world and displays it on the mobile device display.

2.1 SimConnect App: Read information from Microsoft Flight Simulator

The first application runs on the PC of a user playing MSFS. Using the FS2020 SDK, we developed a separate application that connects to the MSFS, extracts at a defined framerate the latitude, longitude and altitude information of the virtual airplane and sends it via a UDP protocol to our server.

2.2 Server

The server receives the latitude, longitude and altitude information of the virtual airplane from the first application and sends it, via UDP protocol, to any connected clients.

2.3 Mobile AR application on cellphone

The mobile device running the application (Figure 2) was an Android phone (Google Pixel 3a, Google, CA, USA), which has sensors including a camera, an inertial measurement unit (IMU), GPS, and a magnetometer. Using the GPS the device measures its real world location. Using ARCore (Google, CA, USA), the device

*Email: cleuze@nakamir.net

camera and IMU are used to map the real world environment including the ground plane around the mobile device and understand its position and movement within this real world environment using simultaneous localization and mapping (SLAM). The magnetometer acts as a compass and allows the device to understand in which direction other real world locations and GPS coordinates are located.

In the mobile AR application we load the same 3D plane model as the virtual airplane used in MSFS. The mobile application connects to the server, receives the latitude, longitude and altitude information of the virtual airplane flying in the MSFS, and calculates the position of the plane model in the mapped 3D space.

To allow smooth movement of the avatar at the actual speed of the virtual airplane in the MSFS, the mobile AR application always performs a spline interpolation between multiple received real world locations to a continuous movement path. The speed of the 3D avatar is calculated based on the time stamp when the GPS coordinates were received. The avatar is then moved along this interpolated path at the calculated speed.

2.4 Mobile application on see-through headset

We also ran the mobile application on a see-through headset. Because the see-through headset we used (Hololens 2, Microsoft, WA, USA) does not have a GPS or magnetometer, we used it in combination with the smart phone from section 2.3. The Hololens 2 performs its own SLAM to map the real world surroundings. The Hololens 2 environmental map and the smartphone map were then synchronized by using an image marker to ensure that the origin and coordinate axes of the Hololens 2 space coincided with the origin and axes of the space mapped by the cell phone app. Once both spaces were synchronized, the movement path of the 3D plane model in the smart phone app space was streamed via a UDP connection to the see-through headset to update the position and movement of another identical 3D plane model on the see-through display.

3 RESULTS & DISCUSSION

We have shown for the first time an example where a virtual model, which exists and moves only in a virtual representation of the world, can also be seen at the actual location in the real world with the help of an AR application. MSFS contains a highly realistic virtual representation of the real world, which was constructed from satellite imagery and 3D city data. A user can freely move within this virtual representation of the real world and real world positioning data like GPS or altitude translates accurately to positions in the virtual world and vice versa. This simple translation of position information from the virtual to the real world makes MSFS a good candidate to translate also the position of objects that only exist in the virtual world to the real world. However, a similar application does not need to be restricted to MSFS but could either work on a different virtual representation of the real world (e.g. google maps) or even only a subset of the real world, like a city or a single venue. It is only important that there is a well-defined one-to-one mapping between a location in the virtual world and the corresponding location in the real world. It is also possible that the presented experience goes both ways, not only from the virtual world to the real world, like presented in this work, but also from the real world to the virtual world like for a Pokemon Go like experience. If streaming from both virtual world to real world and real world to virtual world is implemented, this would allow users in the virtual world and users in the real world to interact and, e.g., perform tasks together that are only possible in one of the worlds.

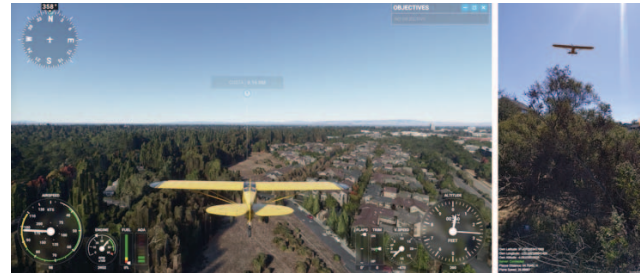


Figure 2: (Left) A view of a 3D model of a virtual airplane flying in Microsoft Flight Simulator 2020. (Right) A view of the mobile app showing the camera stream of the real world and, in the sky, a 3D rendering of the same airplane. The location of the 3D model of the airplane in the app is the same world position as the position of the 3D model in the virtual world of the flight simulator.

Our current application still has some limitations: The movement interpolation always requires several locations, leading to a small delay between the MSFS and real world plane position. This delay can be minimized by increasing the frequency, with which the position information is sent and received and by using less points for the spline curve interpolation.

Realistic occlusion based on depth maps only worked reliably for close-by objects within less than 10m. For this reason, the 3D airplane model currently appears always in front of environmental objects that would normally occlude the airplane. This problem could be solved if the mobile application has a detailed knowledge of the environment and altitude profiles and, thus, can calculate whether a third object like the airplane is visible or not.

The see-through display we used was a Hololens 2 device with an additive light display. When looking at the sky, the daylight background was very bright so that the 3D airplane model was hardly visible in the headset when far away. The 3D airplane model became better visible in the headset when it landed and in front of darker background such as trees or buildings.

In our application we are currently only transmitting GPS and altitude information from the virtual world to the real world user. While this allows to depict a realistic movement path of the airplane, additional information such as yaw, pitch and roll would need to be transmitted for the actual airplane orientation.

If the 3D avatar is not an airplane, it might be useful to transmit additional information about the avatar, which are completely different depending on the type of avatar. For a humanoid avatar, it might be useful to transmit information like emotions, gaze direction or gestures. Gaming applications might benefit from transmitting player statistics of the user in the virtual world such as the player health or high score statistics.

4 CONCLUSION

The presented work is an important example of how AR can be used to connect a virtual world, which is at least in part a representation of the real world, with the real world. While the presented virtual world was a completely self-contained application running on a PC, it was possible by a user in the real world to view the virtual player on a mobile device as if the virtual player is moving within the real world. This was done by accurately mapping the location information and movement between the virtual world and the real world. Such an application can have important implications not only for gaming applications but also for telepresence, social and collaborative applications where a real world user has an interest in interacting with the virtual player even though they are spatially separated.